```
  MockedAuthService
end
```

The purpose of this setup would be that the application uses the class that corresponds to the environment via `AUTH_SERVICE`. In development mode `MockedAuthService` gets autoloaded when the initializer runs. Let's suppose we do some requests, change its implementation, and hit the application again. To our surprise the changes are not reflected. Why?

As we saw earlier, Rails removes autoloaded constants, but `AUTH_SERVICE` stores the original class object. Stale, non-reachable using the original constant, but perfectly functional.

The following code summarizes the situation:

```
class C
  def quack
    'quack!'
  end
end

X = C
Object.instance_eval { remove_const(:C) }
X.new.quack # => quack!
X.name      # => C
C      # => uninitialized constant C (NameError)
```

Because of that, it is not a good idea to autoload constants on application initialization.

In the case above we could implement a dynamic access point:

```
# app/models/auth_service.rb
class AuthService
  if Rails.env.production?
    def self.instance
      RealAuthService
    end
  else
    def self.instance
      MockedAuthService
    end
  end
end
```

and have the application use `AuthService.instance` instead. `AuthService` would be loaded on demand and be autoload-friendly.

10.5 `require_dependency` and Initializers

As we saw before, `require_dependency` loads files in an autoloading-friendly way. Normally, though, such a call does not make sense in an initializer.

One could think about doing some `require_dependency` calls in an initializer to make sure certain constants are loaded upfront, for example as an attempt to address the gotcha with STIs.

Problem is, in development mode autoloaded constants are wiped if there is any relevant change in the file system. If that happens then we are in the very same situation the initializer wanted to avoid!

Calls to `require_dependency` have to be strategically written in autoloaded spots.

10.6 When Constants aren't Missed

10.6.1 Relative References

Let's consider a flight simulator. The application has a default flight model

```
# app/models/flight_model.rb
```